

Rising Waters: Machine Learning-Based Flood Prediction System

Project Description:

Rising Waters is a machine learning-based web application designed to predict flood risk using meteorological and rainfall-related parameters. The system analyzes environmental factors such as temperature, humidity, cloud cover, annual rainfall, seasonal rainfall, average June rainfall, and subdivision information to estimate whether a region has a High Flood Risk or Low Flood Risk.

The application is developed using the Flask framework and integrates a pre-trained XGBoost machine learning model with Scikit-learn preprocessing techniques. Users enter environmental parameters through a simple web interface, and the application instantly predicts the flood risk level.

The project aims to support disaster management authorities, researchers, and local governments in making informed decisions for flood preparedness and mitigation.

Scenarios

Scenario 1

A disaster management officer enters rainfall and weather data collected from a district before the monsoon season. The system predicts High Flood Risk, enabling authorities to issue early warnings and prepare emergency response teams.

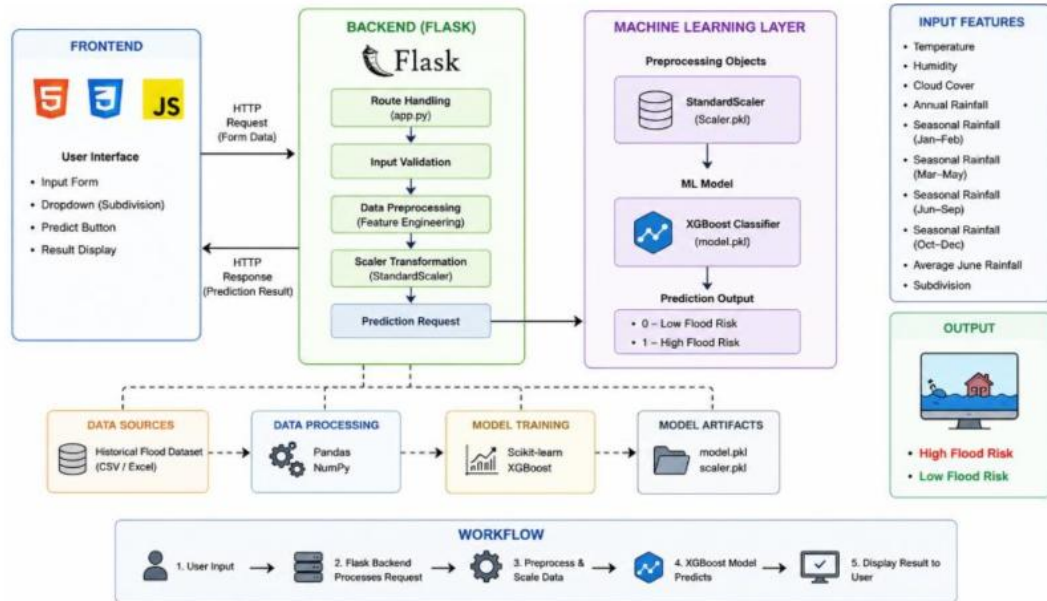
Scenario 2

A researcher analyzes historical rainfall data for a region. After entering the environmental parameters, the application predicts Low Flood Risk, indicating relatively safe conditions.

Scenario 3

A weather analyst receives updated rainfall measurements after continuous heavy rainfall. The new values are entered into Rising Waters, which predicts High Flood Risk, assisting officials in planning evacuation and relief measures.

Technical Architecture:



Description: Rising Waters utilizes a modular three-tier architecture for accurate flood risk prediction. The frontend provides an interactive user interface built with **HTML**, **CSS**, and **JavaScript**, while the **Flask-based backend** manages user input validation, data preprocessing, and communication with the machine learning model. The system uses a trained **XGBoost classifier** with **StandardScaler** to analyze weather and rainfall parameters and predict flood risk. The application is deployed locally using the Flask development server, providing real-time flood risk predictions through a simple web interface.

Pre-requisites:

- 📖 **Flask Framework Knowledge:** <https://flask.palletsprojects.com/>
- 📖 **Python Programming:** <https://docs.python.org/3/>
- 📖 **Machine Learning using Scikit-learn:** <https://scikit-learn.org/stable/>
- 📖 **XGBoost Documentation:** <https://xgboost.readthedocs.io/>
- 📖 **Pandas Documentation:** <https://pandas.pydata.org/docs/>
- 📖 **NumPy Documentation:** <https://numpy.org/doc/>
- 📖 **HTML, CSS, and JavaScript:** <https://www.w3schools.com/>
- 📖 **Version Control with Git:** <https://git-scm.com/doc>
- 📖 **Visual Studio Code:** <https://code.visualstudio.com/docs>
- 📖 **Postman API Testing:** <https://learning.postman.com/docs/>

Project Workflow:

Milestone 1: Data Collection and Model Development

- **Activity 1.1:** Collect and preprocess historical flood and weather datasets.
- **Activity 1.2:** Perform data cleaning, feature selection, and data preprocessing using Pandas and NumPy.
- **Activity 1.3:** Train and evaluate the **XGBoost** machine learning model for flood prediction.
- **Activity 1.4:** Save the trained model and StandardScaler for integration into the web application.

Milestone 2: Backend Development

- **Activity 2.1:** Develop the Flask backend to handle user requests and application routing.
- **Activity 2.2:** Implement data validation, preprocessing, and integration with the trained machine learning model for prediction.

Milestone 3: Application Development

- **Activity 3.1:** Develop the main application logic in **app.py**, including loading the trained model, processing user inputs, generating flood predictions, and displaying results.

Milestone 4: Frontend Development

- **Activity 4.1** Design and develop a responsive user interface using HTML, CSS, and JavaScript for collecting environmental parameters.
- **Activity 4.2:** Create dynamic web pages using Flask's **render_template()** to display flood prediction results interactively.

Milestone 5: Testing and Deployment

- **Activity 5.1:** Perform functional testing using **Postman** and verify prediction accuracy and API responses.
- **Activity 5.2** Prepare the application for local deployment by configuring the Flask server and installing all required dependencies.

Milestone 6: Conclusion

Summarize the development and successful implementation of the Rising Waters flood prediction system, highlighting its ability to predict High Flood Risk and Low Flood Risk using machine learning and its potential to support disaster preparedness and early warning systems.

Milestone 1: Model Selection and Architecture

In this milestone, the focus is on selecting an appropriate machine learning algorithm for flood prediction and designing the overall architecture of the application. This involves collecting and preprocessing historical weather and rainfall data, evaluating different machine learning algorithms, and choosing the most suitable model based on prediction accuracy and performance. The selected model is then integrated with a Flask-based web application to provide real-time flood risk prediction.

Activity 1.1: Dataset Collection and Preparation

Before training the flood prediction model, an appropriate dataset containing historical weather and rainfall information is required. Follow the steps below to prepare the dataset.

- **Step 1 – Collect the Dataset:** Obtain a historical flood prediction dataset containing parameters such as Temperature, Humidity, Cloud Cover, Annual Rainfall, Seasonal Rainfall (Jan–Feb, Mar–May, Jun–Sep, Oct–Dec), Average June Rainfall, and Subdivision.
- **Step 2 – Load the Dataset:** Load the dataset into Python using the Pandas library for preprocessing and analysis.

```
df = pd.read_excel("flooddataset.xlsx")
```

```
df.head()
```

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
0	29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.866667	649.9	0
1	28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.300000	256.4	1
2	28	75	42	3271.2	21.7	336.3	2343.0	570.1	186.200000	308.9	0
3	29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.066667	862.5	0
4	31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.400000	586.9	0

- **Step 3 – Clean the Dataset:** Handle missing values, remove duplicate records, and ensure that all numerical features are correctly formatted before training.
- **Step 4 – Select Features:** Select the required input features and the target variable (Flood Risk) for model training.
- **Step 5 – Split the Dataset:** Divide the dataset into training and testing sets to evaluate the model effectively.
- **Step 6 – Save the Trained Model:** After training the XGBoost classifier and StandardScaler, save them as serialized files (model.pkl and scaler.pkl) for use in the Flask application.

```
import pickle

with open("../models/best_model.pkl", "wb") as f:
    pickle.dump(rf, f)

with open("../models/scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)

print("Model saved successfully!")
```

Model saved successfully!

```
with open("../models/best_model.pkl", "rb") as file:
    model = pickle.load(file)

prediction = model.predict([X_test[0]])

print("Prediction:", prediction[0])
print("Actual Value:", y_test.iloc[0])
```

Prediction: 0
Actual Value: 0

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(rf, X, y, cv=5)

print("Cross Validation Scores:", scores)
print("Average Accuracy:", scores.mean())
```

Cross Validation Scores: [0.95652174 1. 1. 1. 1.]
Average Accuracy: 0.9913043478260869

Important: Ensure that the same preprocessing steps used during training are applied when making predictions through the Flask application to maintain prediction accuracy.

Activity 1.2: Research and Select the Appropriate Machine Learning Model

The following aspects were considered while selecting the most suitable machine learning model for the flood prediction system:

- **Understand the Project Requirements:** Analyze the project objectives and identify the need for a classification model capable of predicting High Flood Risk and Low Flood Risk based on weather and rainfall parameters.
- **Explore Different Machine Learning Algorithms:** Research and compare various classification algorithms such as Logistic Regression, Decision Tree, Random Forest, Support Vector Machine (SVM), and XGBoost for flood prediction.
- **Evaluate Model Performance:** Compare the models using evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and prediction efficiency on the prepared dataset.
- **Select the Optimal Model:** Choose the XGBoost Classifier because it achieved the highest prediction accuracy, effectively handled structured tabular data, reduced overfitting through ensemble learning, and provided fast and reliable flood risk predictions. A StandardScaler was also applied to preprocess the input features before prediction.

	Model	Accuracy
0	Decision Tree	1.000000
1	Random Forest	1.000000
3	XGBoost	1.000000
2	KNN	0.913043

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, illustrating the interaction between the Frontend (HTML, CSS, JavaScript), Flask Backend, Machine Learning Model (XGBoost), and the prediction results.
- **Detail Frontend Functionality:** Describe how users interact with the application by entering environmental parameters such as Temperature, Humidity, Cloud Cover, Annual Rainfall, Seasonal Rainfall (Jan–Feb, Mar–May, Jun–Sep, Oct–Dec), Average June Rainfall, and Subdivision through the web interface.
- **Outline Backend Responsibilities:** Explain how the Flask backend receives user input through the /predict route, validates the submitted values, converts them into a Pandas DataFrame, applies feature scaling using the StandardScaler, and passes the processed data to the trained XGBoost model.
- **Describe Machine Learning Integration:** Define how the backend loads the saved XGBoost model and StandardScaler, preprocesses the input features, performs flood risk prediction, and returns the prediction result (High Flood Risk or Low Flood Risk) to the frontend for display.

Activity 1.4: Set Up the Development Environment

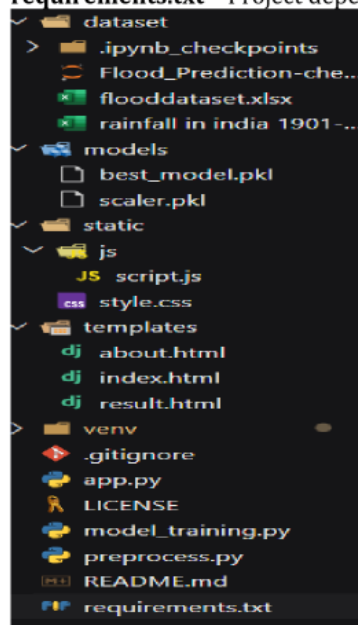
- **Install Python and Pip:** Ensure **Python 3.10 or later** is installed on your system along with **pip** for managing project dependencies.
- **Create a Virtual Environment:** Set up a virtual environment using **venv** to isolate project dependencies:
- **Install Required Libraries:** Install all required dependencies using the **requirements.txt** file.

```
PS C:\Users\K RAVI KUMAR\Desktop\Rising-Waters>python -m venv myenv
(venv) PS C:\Users\K RAVI KUMAR\Desktop\Rising-Waters>pip install -r
requirements.txt
```

Verify Required Packages: Ensure that essential libraries such as Flask, Pandas, NumPy, Scikit-learn, XGBoost, Joblib, and OpenPyXL are installed successfully.

Set Up the Project Structure: Organize the project into appropriate folders and files, including:

- **templates/** – HTML pages
- **static/** – CSS, JavaScript, and images
- **model/** – Trained model (model.pkl) and scaler (scaler.pkl)
- **app.py** – Main Flask application
- **requirements.txt** – Project dependencies



Run the Application: Start the Flask development server and verify that the application is running correctly

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Flood Prediction Features

Implement the core functionalities required for flood risk prediction using the trained machine learning model.

- **User Input Module:** Develop a web form that allows users to enter environmental parameters such as Temperature, Humidity, Cloud Cover, Annual Rainfall, Seasonal Rainfall (Jan–Feb, Mar–May, Jun–Sep, Oct–Dec), Average June Rainfall, and Subdivision.
- **Data Preprocessing Module:** Validate the user inputs, convert them into the required numerical format, create a Pandas DataFrame, and apply the trained StandardScaler before prediction.
- **Flood Prediction Module:** Load the trained XGBoost model, process the scaled input data, and predict the flood risk level as either High Flood Risk or Low Flood Risk.
- **Result Display Module:** Display the prediction result on the web page in a clear and user-friendly format with appropriate visual indicators.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask

- Configure routes in app.py for the home page (/) and the flood prediction endpoint (/predict).

Process User Input

- Create an input form in index.html to collect weather and rainfall parameters from the user.
- Use Flask's request.form to retrieve the submitted form data.
- Validate that all required fields are provided and convert them into numerical values before processing.

Integrate the Machine Learning Model

- Load the trained XGBoost model (model.pkl) and StandardScaler (scaler.pkl) during application startup.
- Convert the user inputs into a Pandas DataFrame with the correct feature names.
- Apply the StandardScaler to normalize the input features.
- Pass the processed data to the trained model to predict the flood risk.
- Display the prediction result (High Flood Risk or Low Flood Risk) on the result page using Flask's render_template() function.

Milestone 3: App.py Development

Milestone 3 focuses on developing and refining the core logic of the app.py file, which serves as the backbone of the Rising Waters application. In this milestone, Flask routes are implemented to handle user requests, process weather and rainfall data, integrate the trained machine learning model, and generate flood risk predictions. Functional testing is also performed to ensure the application produces accurate and reliable results.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py

Establish the main routes for the application, each serving a specific purpose:

- / — Home page route that renders the index.html interface where users enter environmental parameters.
- /predict — POST endpoint that receives user input, preprocesses the data, invokes the trained machine learning model, and displays the prediction result.

```
@app.route("/predict", methods=["POST"])
def predict():
    try:
        # Read values from the form
        features = {
            "Temp": float(request.form["Temp"]),
            "Humidity": float(request.form["Humidity"]),
            "Cloud Cover": float(request.form["Cloud-Cover"]),
            "ANNUAL": float(request.form["ANNUAL"]),
            "Jan-Feb": float(request.form["Jan-Feb"]),
            "Mar-May": float(request.form["Mar-May"]),
            "Jun-Sep": float(request.form["Jun-Sep"]),
            "Oct-Dec": float(request.form["Oct-Dec"]),
            "avgjune": float(request.form["avgjune"]),
            "sub": float(request.form["sub"])
        }
        # Create DataFrame with correct column names
        data = pd.DataFrame([features])
        # Scale the input
        data_scaled = scaler.transform(data)

        # Predict
        prediction = model.predict(data_scaled)

        if prediction[0] == 1:
            result = "⚠️ High Chance of Flood"
        else:
            result = "✅ Low Chance of Flood"

        return render_template("result.html", prediction=result)

    except Exception as e:
        return f"Error: {e}"
```

Set Up Route Handlers

- Implement the /predict route to retrieve weather and rainfall parameters using request.form.
- Validate that all required input fields are provided and convert them into numerical values.
- Create a Pandas DataFrame with the input features required by the trained model.
- Apply the saved StandardScaler to preprocess the input data before prediction.

```
from flask import Flask, render_template, request
import pickle
import pandas as pd

app = Flask(__name__)

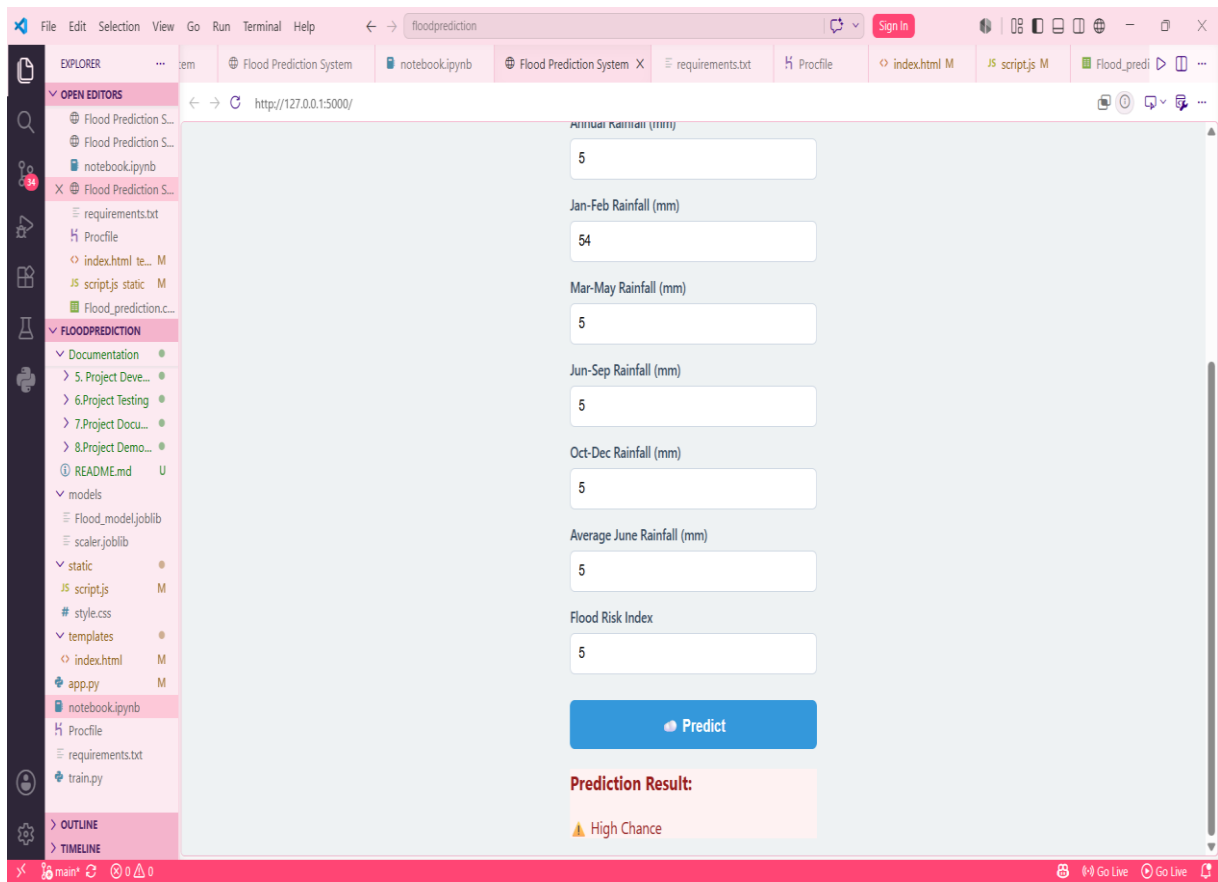
# Load model and scaler
model = pickle.load(open("models/best_model.pkl", "rb"))
scaler = pickle.load(open("models/scaler.pkl", "rb"))

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/about")
def about():
    return render_template("about.html")
```

Integrate the Machine Learning Model

- Load the trained XGBoost model (model.pkl) and StandardScaler (scaler.pkl) during application startup.
- Pass the scaled input data to the trained model for prediction.
- Interpret the prediction result:
 - 1 → High Flood Risk
 - 0 → Low Flood Risk
- Return the prediction result to the user by rendering result.html.



Milestone 4: Frontend Development

Milestone 4 focuses on designing and developing a responsive, user-friendly interface for the Rising Waters application. The frontend enables users to enter weather and rainfall parameters easily and view flood prediction results in a simple and interactive manner. The interface is developed using HTML, CSS, and JavaScript, ensuring an intuitive user experience across different devices.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure

- Develop the main HTML page (index.html) containing the application header, navigation section, flood prediction form, and prediction result section.
- Use semantic HTML elements to create a well-structured and accessible web interface.
- Include labels, placeholders, and input fields for all required weather and rainfall parameters.

Design a Responsive Layout Using CSS

- Create a CSS stylesheet (static/css/style.css) to design a clean, modern, and responsive user interface.
- Use Flexbox and CSS Grid to organize the input form and prediction result sections.
- Apply responsive media queries to ensure the application works effectively on desktops, tablets, and mobile devices.

Develop User Input Components

- Create input fields for:
 - Temperature
 - Humidity
 - Cloud Cover
 - Annual Rainfall
 - Seasonal Rainfall (Jan–Feb, Mar–May, Jun–Sep, Oct–Dec)
 - Average June Rainfall
 - Subdivision
- Add a Predict Flood Risk button to submit the form.
- Design a dedicated result section to display High Flood Risk or Low Flood Risk with appropriate visual styling.

Suggested Screenshots

- Home page (index.html)
- Input form
- Responsive interface

Activity 4.2: Creating Dynamic Content Rendering

Handle User Input

- Implement form submission to collect weather and rainfall parameters from the user.
- Validate that all required fields contain valid numerical values before submitting the data.

Display Prediction Results

- Submit the form data to the Flask backend through the /predict route.
- Display the flood prediction result dynamically on the result.html page.
- Clearly indicate whether the predicted result is High Flood Risk or Low Flood Risk.

Improve User Experience

- Display prediction results in a clear and readable format.
- Ensure smooth navigation between the input page and the result page.
- Maintain a responsive layout for different screen sizes and devices.

Milestone 5: Deployment

In Milestone 5, the focus is on preparing the Rising Waters application for local deployment and verifying its functionality. This involves configuring the development environment, installing all required dependencies, launching the Flask server, and testing the flood prediction system to ensure accurate and reliable results.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment

- Open Command Prompt or PowerShell.
- Navigate to the project directory.
- Create and activate a virtual environment.

```
python -m venv venv  
venv\Scripts\activate
```

Install Required Dependencies

Install all project dependencies using the **requirements.txt** file.

```
pip install -r requirements.txt
```

Project Configuration

- Ensure that the trained machine learning model (model.pkl) and preprocessing scaler (scaler.pkl) are available in the project directory.
- Verify that all project files, including the templates and static folders, are correctly organized before running the application.

Activity 5.2: Local Testing and Verification

Start the Flask Development Server

Run the application using:

```
python app.py
```

Access the Application

Open a web browser and navigate to:

```
http://127.0.0.1:5000
```

Verify the Application

- Enter valid weather and rainfall parameters.
- Submit the prediction request.
- Verify that the application correctly displays either High Flood Risk or Low Flood Risk.
- Perform functional testing using Postman to verify the /predict endpoint.
- Confirm that the application returns HTTP 200 OK for valid requests and handles invalid inputs appropriately.

Activity 5.3: Deployment (Optional)

If the application is deployed on a cloud platform such as Render, Railway, or PythonAnywhere, include the deployment URL and verify that the application is accessible through a web browser.